UPSC EPFO · APFC RECRUITMENT TEST · FOUNDATION LAYER · MODULE 9 · CHAPTER F9.6

Operating Systems, Software and Programming

Chapter F9.6 — Four More 2023 Items, Including the Ones the Blueprint Warned Would Need Real Hours

Module	**Foundation Layer — Module 9 — General Science & Computer Applications**
Chapter	**Chapter F9.6**
Topic	**Official syllabus head 6 — Basic Knowledge of Computer Applications**
Version	**v1.0**
Compiled	**16 August 2026**
Exam stage	**Recruitment Test (RT)**

NOTE · THIS IS THE CHAPTER THE BLUEPRINT SINGLED OUT

Blueprint §4.4 warned that within Computer Applications the **C, HTML and operating-system items** are the ones that *"need actual targeted preparation. Budget real hours there, not zero."*

Four of the ten 2023 computer questions are in this chapter: the **real-time operating system** (§2.3), **top-down languages** (§4.3), the **C structure operator** (§5.2) and the **hard disk error-check command** (§3).

None of them is guessable. All four are single, specific facts. That combination — unguessable but finite — makes this the best-value chapter in Module 9 per hour spent.

Table of Contents

0. How This Chapter Works

1. Session 1 — Kinds of Software ★★★☆☆

2. Session 2 — The Operating System ★★★★★

- 2.1 What it does
- 2.2 Types of operating system
- 2.3 Real-time operating systems — the 2023 item

3. Session 3 — Command-Line Utilities ★★★★☆

4. Session 4 — Languages, Translators and Design ★★★★★

- 4.1 Generations of language
- 4.2 The three translators
- 4.3 Top-down and bottom-up — the 2023 item

5. Session 5 — The C Language ★★★★★

- 5.1 Background and basics
- 5.2 The operators — and the 2023 item
- 5.3 Control structures

6. Session 6 — Office Applications ★★★☆☆

7. Graded Practice — 25 Questions ★★★★★

- 7.1 Level 1 — Recognition
- 7.2 Level 2 — Understanding
- 7.3 Level 3 — Recruitment Test standard

8. Answers and Explanations

- 8.1 Level 1
- 8.2 Level 2
- 8.3 Level 3

9. One-Minute Revision

10. Five-Minute Wall Sheet

0. How This Chapter Works

STUDY SESSION · CHAPTER F9.6 — SIX SESSIONS

1. **Session 1** — Kinds of software (§1) ★★★☆☆
2. **Session 2** — The operating system, and **real-time systems** (§2) ★★★★★ **2023**
3. **Session 3** — **Command-line utilities** (§3) ★★★★☆ **2023**
4. **Session 4** — Languages, translators and **top-down design** (§4) ★★★★★ **2023**
5. **Session 5** — **The C language and its operators** (§5) ★★★★★ **2023**
6. **Session 6** — Office applications and shortcuts (§6) ★★★☆☆

Then 25 graded questions in §7. Budget **two and a half days**.

1. Session 1 — Kinds of Software ★★★☆☆

Category	What it is	Examples
System software	Runs and manages the computer itself	**Operating system**, device drivers, **language translators**, utilities
Application software	Performs tasks for the user	Word processor, spreadsheet, browser, DBMS
Utility software	Maintains and protects the system	Antivirus, disk defragmenter, compression, backup
Firmware	Software stored permanently in **ROM**	BIOS

MUST MASTER · WHICH SIDE OF THE LINE

The recurring question is simply "which of the following is system software", and the answer is almost always the **operating system**, with a word processor, spreadsheet and browser as distractors.

The test: does it **serve the machine** (system) or **serve the user's task** (application)? A **language translator** — compiler, interpreter, assembler — is **system** software, which surprises people, because it serves the machine's ability to run programs rather than any user task.

Licensing terms occasionally appear: **freeware** is free to use; **shareware** is free to try; **open source** publishes its source code (Linux); **proprietary** does not (Windows).

CHECK YOURSELF · SESSION 1

1. Classify: operating system, spreadsheet, antivirus, compiler, BIOS
2. Which is system software — a compiler or a browser?
3. Distinguish freeware, shareware and open source

2. Session 2 — The Operating System ★★★★★

2.1 What it does

An **operating system** is system software that acts as an **interface between the user and the hardware** and **manages the computer's resources**.

Functions: process management · memory management · file management · device management · security · scheduling · providing the user interface.

Key terms:

Term	Meaning
Kernel	The **core** of the operating system
Virtual memory	Using disk space to extend apparent main memory
Paging	Dividing memory into fixed-size blocks
Spooling	Queuing jobs — most familiarly **print jobs** — so the CPU need not wait
Deadlock	Two or more processes each waiting for a resource the other holds
Booting	Starting the computer. **Cold boot** = from power off. **Warm boot** = restart while running
GUI / CLI	Graphical user interface versus command-line interface

2.2 Types of operating system

Type	Defining feature
Batch	Jobs collected and run in batches, with **no user interaction**
Time-sharing	CPU time **shared** among many users interactively
Multiprogramming	Several programs held in memory at once
Multiprocessing	**More than one CPU**
Distributed	Multiple **networked** computers acting as one
Network	Manages a network's shared resources
Real-time	Responds within a **guaranteed deadline** — see below

2.3 Real-time operating systems — the 2023 item

MUST MASTER · WHAT MAKES AN OS "REAL-TIME"

A **real-time operating system (RTOS)** is one that must respond to input **within a strict, guaranteed time constraint — a deadline.** Its defining characteristic is not speed as such, but **predictability**: the response time must be bounded and guaranteed.

Two kinds:

	Hard real-time	**Soft real-time**
Missing a deadline	**Unacceptable — a system failure**	Undesirable but tolerable
Examples	Missile guidance, air traffic control, medical life support, industrial robots, anti-lock brakes	Video streaming, online transaction systems

Typical applications: air traffic control, missile and flight control, medical life-support equipment, industrial process control, embedded systems.

Two traps to expect:

1. **"Fast" is not the definition.** An RTOS guarantees a **deadline**; an ordinary OS may be quicker on average but guarantees nothing. Options describing it as simply "very fast" are wrong
2. **Windows, Linux and macOS are general-purpose, NOT hard real-time.** An option naming a desktop OS as a real-time system is wrong — and it is tempting, because they feel responsive

CHECK YOURSELF · SESSION 2

1. Define an operating system. Name five of its functions
2. What is the kernel? What is spooling most associated with?
3. Distinguish cold boot from warm boot
4. What defines a real-time OS — speed, or something else?
5. Distinguish hard from soft real-time, with an example of each
6. Is Windows a hard real-time operating system?

3. Session 3 — Command-Line Utilities ★★★★☆

The 2023 paper asked which command **checks a hard disk for errors**. Learn the small set below.

Command	What it does
CHKDSK	**Checks the disk for errors** and displays a status report. **The 2023 answer**
`SCANDISK`	The older equivalent utility
`DIR`	Lists the contents of a directory
`COPY`	Copies files
`DEL`	Deletes files
`MD` or `MKDIR`	Makes a directory
`CD`	Changes the current directory
`RD`	Removes a directory
`FORMAT`	Prepares a disk for use, **erasing existing data**
`REN`	Renames a file
`CLS`	Clears the screen
`ATTRIB`	Displays or changes file attributes
`DEFRAG`	Defragments a disk

CHKDSK checks; FORMAT erases. Do not confuse the two

MUST MASTER · CHKDSK CHECKS, FORMAT DESTROYS

CHKDSK — short for "check disk" — scans a disk for **errors** in the file system and bad sectors, and reports on them. It is the answer to any question about checking a disk's integrity.

FORMAT prepares a disk for use and in doing so **erases its contents**. It is the destructive one, and it is the distractor offered beside CHKDSK.

A secondary distinction that sometimes appears: **internal** commands are built into the command interpreter (`DIR` , `COPY` , `CLS` , `DEL` , `CD`); **external** commands exist as separate program files (`CHKDSK` , `FORMAT` , `DEFRAG`).

CHECK YOURSELF · SESSION 3

1. Which command checks a disk for errors? Which one erases it?
2. What do `DIR`, `REN`, `CLS` and `ATTRIB` do?
3. Distinguish internal from external commands, with two examples of each

4. Session 4 — Languages, Translators and Design ★★★★★

4.1 Generations of language

Generation	Type	Examples
1GL	**Machine language** — binary	—
2GL	**Assembly language** — mnemonics	—
3GL	**High-level** procedural	**C**, FORTRAN, COBOL, BASIC, Pascal
4GL	Query and database languages	**SQL**
5GL	Artificial intelligence languages	Prolog, LISP

Low-level means machine and assembly language — close to the hardware. **High-level** means human-readable and portable.

4.2 The three translators

Translator	What it does	Consequence
Compiler	Translates the **entire program at once** into object code	**Faster execution**; errors reported together. C, C++
Interpreter	Translates and executes **line by line**	**Easier debugging**; slower execution. Python, BASIC
Assembler	Converts **assembly language** into machine code	—

All three are **system** software (§1).

4.3 Top-down and bottom-up — the 2023 item

MUST MASTER · WHICH DIRECTION IS WHICH, AND WHICH LANGUAGES

	Top-down	Bottom-up
Direction	Start with the **whole problem** and **break it into sub-problems**	Start with the **smallest components** and **combine** them
Technique	**Stepwise refinement**	Composition
Associated with	**Structured / procedural programming**	**Object-oriented programming**
Languages	**C, Pascal, FORTRAN**	**C++, Java**

So the 2023 item on "top-down languages" points at the **structured procedural languages — C and Pascal.**

The trap is the reversal, and it is easy to fall for because both descriptions sound constructive. Anchor it this way: **top-down starts at the TOP, meaning the whole problem, and decomposes downwards.** Bottom-up starts with the parts. Object-oriented design builds objects first and assembles them, which is why it is bottom-up.

Errors: a **syntax** error breaks the rules of the language and is caught by the translator; a **logical** error produces a wrong answer from valid code; a **runtime** error occurs during execution.

An **algorithm** is a finite sequence of steps; a **flowchart** is its diagrammatic form; **pseudocode** is its plain-language form.

CHECK YOURSELF · SESSION 4

1. Name the five generations of language with an example of each
2. Compare compiler and interpreter on translation unit, speed and debugging
3. Which approach uses stepwise refinement, and which is object-oriented design?
4. Which languages are associated with top-down design?
5. Distinguish syntax, logical and runtime errors

5. Session 5 — The C Language ★★★★★

5.1 Background and basics

C was developed by **Dennis Ritchie** at Bell Laboratories in **1972**. It is a **procedural, structured** language, often called **middle-level** because it combines high-level constructs with low-level memory access.

- **Case-sensitive** — `Total` and `total` are different identifiers
- Every statement ends with a **semicolon**
- Execution begins at `main()`
- Basic data types: `int`, `char`, `float`, `double`, `void`
- Input and output through `scanf` and `printf`

5.2 The operators — and the 2023 item

Category	Operators	
Arithmetic	`+` `-` `*` `/` and `%` (modulus, giving the remainder)	
Relational	`<` `>` `<=` `>=` `==` (equal to) `!=`	
Logical	`&&` (and) `	`(or)` !` (not)
Assignment	`=`	
Increment and decrement	`++` `--`	
Member access	`.` (dot) and `->` (arrow)	
Others	`sizeof`, the ternary `?:`	

MUST MASTER · THE DOT AND THE ARROW — THE STRUCTURE OPERATORS

A **structure** in C, declared with the keyword `struct`, groups variables of **different data types** under a single name. Its members are reached with one of two operators, and the distinction is the whole question:

Operator	Used when you have	Example
`.` **— the dot operator**	A **structure variable** itself	`employee.salary`
`->` **— the arrow operator**	A **POINTER** to a structure	`ptr->salary`

Dot for a variable, arrow for a pointer. That is the entire rule, and the 2023 question turns on it.

Note the near-miss distractors: `*` is multiplication and also pointer dereferencing, `&` is the address-of operator, and `::` does not exist in C at all — it belongs to C++.

One related distinction worth holding: a **structure** gives each member its own memory, while a **union** makes all members **share** the same memory location.

5.3 Control structures

Decision: `if`, `if-else`, nested `if`, `switch`. **Loops:** `for`, `while`, and `do-while` — of which **only `do-while` is guaranteed to execute at least once**, because its condition is tested at the end. **Jump:** `break` exits the loop; `continue` skips to the next iteration.

Object-oriented concepts, for recognition: class, object, **encapsulation**, **inheritance**, **polymorphism**, abstraction. C is **not** object-oriented; **C++ and Java** are.

CHECK YOURSELF · SESSION 5

1. Who developed C, where and when? Is it case-sensitive?
2. What does the `%` operator return?
3. Which operator accesses a structure member through a pointer? Which through the variable itself?
4. Distinguish a structure from a union
5. Which loop always executes at least once, and why?
6. Is C object-oriented?

6. Session 6 — Office Applications ★★★☆☆

Application	Default extension	Notes
Word processor	`.docx`	Text documents
Spreadsheet	`.xlsx`	**A formula begins with** `=`
Presentation	`.pptx`	Slides

Spreadsheet essentials: a **cell** is identified by its column letter and row number, such as B7. Common functions are `SUM`, `AVERAGE`, `COUNT`, `MAX`, `MIN` and `VLOOKUP`. A **relative** reference changes when copied; an **absolute** reference is fixed using the `$` sign, as in `$B$7`.

Shortcuts worth knowing: Ctrl-C copy · Ctrl-X cut · Ctrl-V paste · Ctrl-Z undo · Ctrl-Y redo · Ctrl-S save · Ctrl-P print · Ctrl-A select all · Ctrl-F find.

CHECK YOURSELF · SESSION 6

1. What character begins a spreadsheet formula?
2. Distinguish relative from absolute cell references. What symbol makes a reference absolute?
3. Give the shortcuts for undo, redo, select all and find

7. Graded Practice — 25 Questions ★★★★★

7.1 Level 1 — Recognition

Q1. An operating system is (a) application software (b) hardware (c) system software (d) firmware only

Q2. Which one of the following is **system** software? (a) An operating system (b) A spreadsheet (c) A web browser (d) A word processor

Q3. The core of an operating system is called the (a) shell (b) BIOS (c) device driver (d) kernel

Q4. Which one of the following operating systems is open source? (a) Windows (b) Linux (c) macOS (d) MS-DOS

Q5. The command used to check a disk for errors and display a status report is (a) `DIR` (b) `FORMAT` (c) `CHKDSK` (d) `DEL`

Q6. The C programming language was developed by (a) Dennis Ritchie (b) James Gosling (c) Bjarne Stroustrup (d) Charles Babbage

Q7. Which translator converts an entire program at once? (a) An interpreter (b) An assembler (c) A loader (d) A compiler

Q8. Machine language is a (a) high-level language (b) first-generation language (c) fourth-generation language (d) query language

Q9. In C, a statement is terminated by a (a) colon (b) full stop (c) semicolon (d) comma

7.2 Level 2 — Understanding

Q10. A real-time operating system is characterised by (a) responding within a strict, guaranteed time constraint (b) batching jobs together without user interaction (c) sharing processor time among many users without any deadline (d) running across multiple networked computers

Q11. Which one of the following is a typical application of a real-time operating system? (a) Word processing (b) Spreadsheet calculation (c) Web browsing (d) Air traffic control

Q12. The top-down approach to program design proceeds by (a) combining small modules into progressively larger ones (b) breaking the overall problem into smaller sub-problems (c) writing machine code before designing (d) testing the program before designing it

Q13. Top-down, structured programming is characteristically associated with (a) object-oriented languages such as Java (b) query languages such as SQL (c) procedural languages such as C and Pascal (d) assembly language only

Q14. In C, the dot operator is used to (a) access a member of a structure variable (b) access a member through a pointer to a structure (c) perform multiplication (d) terminate a statement

Q15. In C, a member of a structure is accessed **through a pointer** using the (a) dot operator (b) ampersand (c) asterisk (d) arrow operator

Q16. Spooling is associated mainly with (a) allocating memory to processes (b) queuing print jobs (c) extending main memory using disk space (d) detecting deadlock

Q17. An interpreter differs from a compiler in that an interpreter (a) always produces faster-executing code (b) produces a separate object file (c) translates and executes the program line by line (d) is used only for machine language

Q18. In a spreadsheet, a formula begins with (a) an equals sign (b) a plus sign (c) an apostrophe (d) a hash sign

7.3 Level 3 — Recruitment Test standard

Q19. Which one of the following statements is **not** correct? (a) A compiler translates the whole program before execution begins (b) An interpreter generally makes debugging easier (c) The kernel is the core of the operating system (d) In a hard real-time system, missing a deadline is acceptable

Q20. Consider the following statements:

1. A real-time operating system must respond within a defined deadline.
2. In a hard real-time system the deadline must be met absolutely.
3. Windows is an example of a hard real-time operating system.

Which of the above statements are correct? (a) 1 and 3 only (b) 1 and 2 only (c) 2 and 3 only (d) 1, 2 and 3

Q21. Match List-I with List-II and select the correct answer using the code given below:

List-I	List-II
A. Compiler	1. Translates the entire program at once
B. Interpreter	2. Translates and executes line by line
C. Assembler	3. Converts assembly language into machine code
D. Kernel	4. The core of the operating system

(a) A-2 B-1 C-3 D-4 (b) A-1 B-2 C-4 D-3 (c) A-1 B-2 C-3 D-4 (d) A-3 B-4 C-1 D-2

Q22. Consider the following statements about the C language:

1. The dot operator accesses a member of a structure variable.
2. The arrow operator accesses a member of a structure through a pointer.
3. C is case-insensitive.

Which of the above statements are correct? (a) 1 and 2 only (b) 2 and 3 only (c) 1 and 3 only (d) 1, 2 and 3

Q23. SQL belongs to which generation of programming languages? (a) First (b) Second (c) Third (d) Fourth

Q24. Which one of the following statements is **not** correct? (a) The top-down approach is associated with structured programming (b) The bottom-up approach begins by breaking the problem into sub-problems (c) Object-oriented design is generally associated with the bottom-up approach (d) Stepwise refinement is a feature of top-down design

Q25. Restarting a computer that is already switched on, without interrupting the power supply, is called (a) a cold boot (b) formatting (c) a warm boot (d) spooling

8. Answers and Explanations

8.1 Level 1

Q	Ans	Explanation
1	c	**System** software — it manages the machine rather than performing a user task
2	a	The **operating system**. The other three are application software
3	d	The **kernel**. The shell is the user-facing interface; the BIOS is firmware
4	b	**Linux.** Windows, macOS and MS-DOS are proprietary
5	c	`CHKDSK` — check disk. `FORMAT`, option (b), is the destructive distractor
6	a	**Dennis Ritchie**, at Bell Laboratories in 1972. Gosling created Java; Stroustrup created C++
7	d	A **compiler** translates the whole program; an interpreter goes line by line
8	b	**First generation.** Assembly is second, high-level third, query languages fourth
9	c	A **semicolon**

8.2 Level 2

Q	Ans	Explanation
10	a	**A strict, guaranteed time constraint.** The defining property is a bounded, predictable deadline — not raw speed. Options (b), (c) and (d) describe batch, time-sharing and distributed systems
11	d	**Air traffic control** — a domain where a late response is a failure. The other three tolerate delay
12	b	**Breaking the overall problem into sub-problems.** Option (a) describes bottom-up
13	c	**Procedural languages such as C and Pascal.** Object-oriented languages go with bottom-up design
14	a	The **dot** operator works on a **structure variable**
15	d	The **arrow** operator works through a **pointer**. Dot for a variable, arrow for a pointer
16	b	**Queuing print jobs**, so the CPU need not wait for a slow printer. Option (c) describes virtual memory
17	c	**Line by line.** That is why debugging is easier but execution slower — so (a) is the reverse
18	a	An **equals sign**

8.3 Level 3

Q	Ans	Explanation
19	d	In a **hard** real-time system, missing a deadline is a **system failure** and is **not** acceptable. That is precisely what distinguishes hard from soft. The other three statements are correct
20	b	**1 and 2 only.** Statement 3 is false — Windows is a **general-purpose** operating system, not a hard real-time one, however responsive it feels
21	c	**A-1 B-2 C-3 D-4** — the four straightforward pairings
22	a	**1 and 2 only.** Statement 3 is false — **C is case-sensitive**, so `Total` and `total` are different identifiers
23	d	**Fourth generation** — query and database languages
24	b	**Reversed.** Breaking the problem into sub-problems is **top-down**, not bottom-up. Bottom-up combines small components upwards. The other three statements are correct
25	c	A **warm boot** — a restart without cutting power. A **cold boot** starts from power off

THINK LIKE UPSC · THE FOUR SHAPES IN THIS CHAPTER

1. **The definition substituted for a property.** Q10 and Q19. An RTOS is defined by a **guaranteed deadline**, not by being fast — and "fast" is the option that feels right
2. **The direction reversal.** Q12 and Q24. **Top-down decomposes; bottom-up composes.** Whenever both appear, check which verb the option uses
3. **The variable-versus-pointer pair.** Q14 and Q15 are the same question twice. **Dot for a variable, arrow for a pointer** — and the two questions sit adjacent to reward knowing both halves
4. **The destructive distractor.** Q5 offers `FORMAT` beside `CHKDSK`. Where one command inspects and another destroys, the exam pairs them

9. One-Minute Revision

ONE-MINUTE REVISION · CHAPTER F9.6

SOFTWARE system = OS, drivers, **translators**, utilities · **application** = word processor, spreadsheet, browser · **firmware** = software in **ROM** (BIOS) · **Linux is open source**, Windows proprietary

OS FUNCTIONS process, memory, file, device management · security · scheduling · user interface · **kernel = the core** · **spooling = queuing PRINT jobs** · virtual memory uses disk to extend RAM · **deadlock** = mutual waiting · **cold boot from power off, warm boot = restart**

OS TYPES batch (no interaction) · time-sharing (CPU shared) · multiprogramming (many programs in memory) · multiprocessing (**many CPUs**) · distributed (networked) · **real-time (guaranteed deadline)**

REAL-TIME OS defined by a **strict guaranteed DEADLINE**, not by speed · **hard** = missing the deadline is a **failure** · **soft** = tolerable · applications: **air traffic control**, missile guidance, life support, industrial control · **Windows is NOT hard real-time**

COMMANDS **`CHKDSK`** **checks a disk for errors** · **`FORMAT`** **erases** · `DIR` lists · `COPY` · `DEL` · `MD` · `CD` · `REN` · `CLS` · `ATTRIB` · `DEFRAG` · internal = built in (`DIR` , `COPY` , `CLS`); external = separate files (`CHKDSK` , `FORMAT`)

LANGUAGE GENERATIONS 1 **machine** · 2 **assembly** · 3 **high-level: C, FORTRAN, COBOL, Pascal** · 4 **SQL** · 5 **AI: Prolog, LISP**

TRANSLATORS **compiler = whole program at once**, faster execution · **interpreter = line by line**, easier debugging · **assembler = assembly to machine code** · all three are **system** software

TOP-DOWN vs BOTTOM-UP **top-down = break the whole into sub-problems, STEPWISE REFINEMENT, structured and procedural, C and Pascal** · **bottom-up = combine components, OBJECT-ORIENTED, C++ and Java**

ERRORS syntax (caught by the translator) · logical (wrong result, valid code) · runtime

C **Dennis Ritchie, Bell Labs, 1972** · procedural, structured, **middle-level** · **case-SENSITIVE** · statements end in a **semicolon** · starts at `main()` · `%` gives the **remainder**

C STRUCTURE OPERATORS **`.`** **dot » a structure VARIABLE** · **`->`** **arrow » a POINTER to a structure** · `&` is address-of, `*` is dereference, `::` is **C++ only** · **structure gives each member its own memory; a UNION shares** memory

LOOPS `for` , `while` , **`do-while`** **executes at least ONCE** · `break` exits, `continue` skips · **C is NOT object-oriented**; C++ and Java are

> **OFFICE** .docx .xlsx .pptx · **a formula begins with** = · $ **makes a reference absolute** · Ctrl-Z undo, Ctrl-Y redo, Ctrl-A select all, Ctrl-F find

10. Five-Minute Wall Sheet

> **FIVE-MINUTE WALL SHEET · CHAPTER F9.6**
>
> **RTOS = GUARANTEED DEADLINE, NOT SPEED · HARD = MISSING IT IS FAILURE · AIR TRAFFIC CONTROL**
>
> **WINDOWS IS NOT REAL-TIME**
>
> **CHKDSK CHECKS · FORMAT ERASES**
>
> **TOP-DOWN = BREAK DOWN = STEPWISE REFINEMENT = C AND PASCAL BOTTOM-UP = BUILD UP = OBJECT-ORIENTED = C++ AND JAVA**
>
> **DOT FOR A VARIABLE · ARROW FOR A POINTER**
>
> **STRUCTURE = SEPARATE MEMORY · UNION = SHARED MEMORY**
>
> **COMPILER WHOLE PROGRAM · INTERPRETER LINE BY LINE (EASIER DEBUGGING)**
>
> **1GL MACHINE · 2GL ASSEMBLY · 3GL C · 4GL SQL · 5GL PROLOG**
>
> **C: RITCHIE, 1972, CASE-SENSITIVE, SEMICOLON, NOT OBJECT-ORIENTED**
>
> **DO-WHILE RUNS AT LEAST ONCE**
>
> **KERNEL = CORE · SPOOLING = PRINT QUEUE · WARM BOOT = RESTART**
>
> **TRANSLATORS ARE SYSTEM SOFTWARE**
>
> **FORMULA STARTS WITH = · $ MAKES IT ABSOLUTE**
>
> **ONE LINE FOR THE INTERVIEW** A real-time operating system is not defined by being fast but by being predictable — it guarantees a bounded response time, which is why an anti-lock brake controller needs one and a desktop computer does not.

> **ACTION · LAST CHAPTER OF MODULE 9**
>
> **Chapter F9.7 — Networking, the Internet and HTML**, covering the final three 2023 items: **IP address validity**, **mesh topology links** and **HTML internal linking**.
>
> Before you move on, state without looking: what defines a real-time OS; which command checks a disk; which design approach uses stepwise refinement; and which operator accesses a structure member through a pointer.